A Mini Project Report on

"SECURE ACCESS WITH HIDDEN PASSWORD ENCRYPTION"

Submitted to

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY, HYDERABAD

In partial fulfilment of the requirement for the award of Marks of

BACHELOR OF TECHNOLOGY

In

"Artificial Intelligence and Data Science"

By

M.Vishal Reddy	(22D41A7265)
M.Mohan	(22D41A7276)
Shaik Sohel	(22D41A72A7)
V.Nagendar Reddy	(22D41A72C1)

Under the guidance of

DR.T.CHARAN SINGH

(Head Of Department)

Department of Artificial Intelligence and Data Science

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(Affiliated to JNTU Hyderabad, Approved by AICTE) Sheriguda (V), Ibrahimpatnam (M), Ranga Reddy Dist.-501 510 (2024-2025)

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC, Accredited by NBA, Affiliated to JNTUH)

DEPARTMENT of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that the project work report entitled **"SECURE ACCESS WITH HIDDEN PASSWORD ENCRYPTION"** which is being submitted by **M. VISHAL REDDY (22D41A7265), M. MOHAN (22D41A7276), SHAIK. SOHEL (22D41A72A7), V. NAGENDAR REDDY (22D41A72C1)** in partial fulfilment for the award of the Marks of **BACHELOR OF TECHNOLOGY** in **Artificial Intelligence & Data Science** of **SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY, HYDERABAD,** is a record of the Bonafide work carried out by them under our guidance and supervision.

INTERNAL GUIDE

DR.T.CHARAN SINGH
(Head Of Department)

HOD

Dr.T.CHARAN SINGH

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

With great pleasure we want to take this opportunity to express our heartfelt gratitude to all the people who helped in making this project work a success. We thank the almighty forgiving us for the courage & perseverance in completing the project.

We are thankful to principal **Prof. Dr. G. Suresh**, for giving us the permission to carry out this project and for providing necessary infrastructure and labs.

We are highly indebted to, **Dr.T.CHARAN SINGH,** Head of the Department of Artificial Intelligence and Data Science, Project Guide, for providing valuable guidance at every stage of this project.

We are grateful to our internal project guide, **DR.T.Charan Singh** for his constant motivation and guidance given by him during the execution of this project work.

We would like to thank the Teaching & Non-Teaching staff of the Department of Artificial Intelligence and Data Science for sharing their knowledge with us. Last but not the least we express our sincere thanks to everyone who helped directly or indirectly for the completion of this project.

trial2

M. Vishal Reddy **(22D41A7265)**

M. Mohan **(22D41A7276)**

Shaik Sohel **(22D41A72A7)**

V. Nagendar Reddy **(22D41A72C1)**

INDEX

Topic	Page No's
CHAPTER-1: INTRODUCTION	**1-2**
CHAPTER-2: LITERATURE SURVEY	**3-4**
CHAPTER-3: SYSTEM ANALYSIS	**5-8**
3.1 Existing System	
3.2 Proposed System	
CHAPTER-4: SYSTEM REQUIREMENTS	**9**
4.1 Hardware Requirements	
4.2 Software Requirements	
CHAPTER-5: SYSTEM STUDY	**10-11**
5.1 Feasibility Study	
5.2 Feasibility Analysis	
CHAPTER-6: SYSTEM ARCHITECTURE	**12-22**
6.1 Data Flow Diagram	
6.2 UML Diagrams	
CHAPTER-7: INPUT AND OUTPUT DESIGN	**23**
7.1 Input Design	
7.2 Output Design	
CHAPTER-8: IMPLEMENTATION	**24**
8.1 MODULES	
8.1.1 Module Description	

Approved trial 2

CHAPTER-9: SOFTWARE ENVIRONMENT **25-40**

9.1 JAVA

9.2 Source Code

CHAPTER-10: RESULTS/DISCUSSIONS **40-50**

10.1 System Test

10.2 Output Screens

CHAPTER-11: CONCLUSION **51**

CHAPTER-12: REFERENCES/BIBLIOGRAPHY **52**

LIST OF FIGURES

S.NO	TABLES/FIGURES	PAGE NO'S
1	6.1 System Architecture	12
	6.2 UML Diagrams	
	6.2.1 Use Case Diagrams	14-15
	6.2.2 Class Diagram	16
	6.2.3 Sequence Diagram	17
	6.2.4 Collaboration Diagram	18
	6.2.5 Activity Diagram	19
	6.2.6 Component Diagram	20
	6.2.7 Deployment Diagram	21
	6.2.8 ER Diagram	22
2	9.2 JAVA Source code with compilation	
	9.2.1 Java Program Language	26
	9.2.2 Java IDE	28
	9.2.3 TCP/IP stack	32
	9.2.4 Tomcat Web server	35
3	10.2 Screenshots	
	10.2.1 Home page	45
	10.2.2 Client Registration Page	45
	10.2.3 Client Login Page	46

	10.2.4 Unauthorized Client Home Page	46
	10.2.5 Web-server Home Page	46
	10.2.6 All Clients Page	47
	10.2.7 Authorized client home page	47
	10.2.8 Password verification page	47
	10.2.9 Owner data upload page	48
	10.2.10 Permissions page	48
	10.2.11 All clients and transactions page	48
	10.2.12 Attacked files page	49
	10.2.13 All details page	49
	10.2.14 Attacker details	49
	10.2.15 File safety score	50

ABSTRACT

Secure password storage is a vital aspect in systems based on password authentication, which is still the most widely used authentication technique, despite its some security flaws. In this paper, we propose a password authentication framework that is designed for secure password storage and could be easily integrated into existing authentication systems. In our framework, first, the received plain password from a client is hashed through a cryptographic hash function (e.g., SHA-256). Then, the hashed password is converted into a negative password. Finally, the negative password is encrypted into an Encrypted Negative Password (abbreviated as ENP) using a symmetric-key algorithm (e.g., AES), and multi-iteration encryption could be employed to further improve security. The cryptographic hash function and symmetric encryption make it difficult to crack passwords from ENPs. Moreover, there are lots of corresponding ENPs for a given plain password, which makes precomputation attacks (e.g., lookup table attack and rainbow table attack) infeasible. The algorithm complexity analyses and comparisons show that the ENP could resist lookup table attack and provide stronger password protection under dictionary attack. It is worth mentioning that the ENP does not introduce extra elements (e.g., salt); besides this, the ENP could still resist precomputation attacks. Most importantly, the ENP is the first password protection scheme that combines the cryptographic hash function, the negative password and the symmetric-key algorithm, without the need for additional information except the plain password.

Keywords: SHA-256, Negative Password, Encrypted Negative Password (ENP), Symmetric-Key Algorithm.

CHAPTER - 1

INTRODUCTION

OWING to the development of the Internet, a vast number of online services have emerged, in which password authentication is the most widely used authentication technique, for it is available at a low cost and easy to deploy . Hence, password security always attracts great interest from academia and industry . Despite great research achievements on password security, passwords are still cracked since users' careless behaviors . For instance, many users often select weak passwords; they tend to reuse the same passwords in different systems; they usually set their passwords using familiar vocabulary for its convenience. In addition, system problems may cause password compromises. It is very difficult to obtain passwords from high security systems. On the one hand, stealing authentication data tables (containing usernames and passwords) in high security systems is difficult. On the other hand, when carrying out an online guessing attack, there is usually a limit to the number of login attempts.

However, passwords may be leaked from weak systems. Vulnerabilities are constantly being discovered, and not all systems could be timely patched to resist attacks, which gives adversaries an opportunity to illegally access weak systems.In fact, some old systems are more vulnerable due to their lack of maintenance. Finally, since passwords are often reused, adversaries may log into high security systems through cracked passwords from systems of low security.

After obtaining authentication data tables from weak systems, adversaries can carry out offline attacks. Passwords in the authentication data table are usually in the form of hashed passwords. However, because processor resources and storage resources are becoming more and more abundant, hashed passwords cannot resist precomputation attacks, such as rainbow table attack and lookup table attack.

Note that there is a trend of generalization of adversaries, because anyone could obtain access to information on vulnerabilities from vulnerability databases, such as the Open Source Vulnerability Database (OSVDB), National Vulnerability Database (NVD), and the Common Vulnerabilities and Exposures (CVE) , and then make use of these information to crack systems. Moreover, they could download and use attack tools without the need for very professional security knowledge. Some powerful attack tools, such as hashcat , RainbowCrack and John the Ripper , provide a variety of functions, such as multiple hash algorithms, multiple attack models, multiple operating systems, and multiple platforms, which raises a higher

demand for secure password storage.I

In these situations, attacks are usually carried out as follows. First, adversaries precompute a lookup table, where the keys are the hash values of elements in a password list containing frequently-used passwords, and the records are the corresponding plain passwords in the password list. Next, they obtain an authentication data table from low security systems. Then, they search for the plain passwords in the lookup table by matching hashed passwords in the authentication data table and the keys in the lookup table. Finally, the adversaries log into higher security systems through cracked usernames and passwords, so that they could steal more sensitive information of users and obtain some other benefits. A considerable number of attacks are carried out in this way, so that adversaries could obtain passwords at a low cost, which is advantageous to their goals.

CHAPTER- 2

LITERATURE STUDY

TITLE: Password Authentication Using Encrypted Negative Password

This paper proposes a novel password protection scheme called **ENP (Encrypted Negative Passwords)**. ENP stores authentication data in a negative format that cannot be reversed, enhancing security against lookup table and dictionary attacks. Unlike salted passwords or key-stretching techniques, ENP does not require additional data elements and offers high resistance to credential leaks. The framework supports secure authentication without sacrificing usability or increasing system complexity. This method serves as the foundation for developing more secure authentication systems in web applications.

TITLE:Enhancing Password Security With Negative Databases

Negative databases (NDBs) aim to store all data except the actual sensitive data, making it difficult for attackers to guess or retrieve the original information. This research applies the concept to passwords, showing how NDBs can hide actual password values and reject unauthorized matches. The authors propose an efficient algorithm for negative database generation and show how it can be used in login systems. This technique aligns closely with the ENP model, as both aim to protect passwords through non-reversible storage.

TITLE: Mitigating Credential Theft in Web Applications Using Lightweight Encryption Techniques

This paper investigates various encryption techniques used to protect login credentials in web apps. The authors introduce lightweight symmetric encryption and one-time hashing to reduce computation costs while ensuring data confidentiality. A case study is presented showing how such encryption can be integrated into login systems and web servers. The authors suggest combining these methods with authentication frameworks like ENP for better protection against injection and replay attacks.

TITLE: Comparative Study of Authentication Schemes Against Brute Force and Dictionary Attacks

This comparative study analyzes the effectiveness of various authentication schemes — including salted hashes, key-stretching, biometric verification, and **ENP** — in resisting offline attacks. The findings reveal that while traditional hashing techniques are fast, they lack resistance against precomputed attacks. In contrast, ENP offers **non-reversible password protection** without extra storage costs or system delays. The paper recommends hybrid models combining ENP with other techniques such as multi-factor authentication for maximum security.

TITLE: *A Survey on Password Storage Mechanisms: Security and Performance*

This study compares the performance and security of widely used password storage mechanisms such as hashing, salting, bcrypt, scrypt, and honeywords. It evaluates their resilience against brute-force and rainbow table attacks. The authors highlight the limitations of traditional methods and propose exploring alternatives like **negative databases** and **hidden password techniques**. The paper concludes that encryption-based techniques can significantly enhance password protection while maintaining performance and user experience.

CHAPTER – 3

SYSTEM ANALYSIS

EXISTING SYSTEM

Hashed Password: The simplest scheme to store passwords is to directly store plain passwords. However, this scheme presents a problem that once adversaries obtain the authentication data table, all passwords are immediately compromised. To safely store passwords, a common scheme is to hash passwords using a cryptographic hash function , because it is infeasible to directly recover plain passwords from hashed passwords. The cryptographic hash function quickly maps data of arbitrary size to a fixed-size sequence of bits. In the authentication system using the hashed password scheme, only hashed passwords are stored. However, hashed passwords cannot resist lookup table attack . Furthermore, rainbow table attack is more practical for its space-time tradeoff . Processor resources and storage resources are becoming richer, which makes the precomputed tables used in the above two attacks sufficiently large, so that adversaries could obtain a higher success rate of cracking hashed passwords.

Salted Password: To resist precomputation attacks, the most common scheme is salted password . In this scheme, the concatenation of a plain password and a random data (called salt) is hashed through a cryptographic hash function. The salt is usually generated at random, which ensures that the hash values of the same plain passwords are almost always different. The greater the size of the salt is, the higher the password security is. However, under dictionary attack, salted passwords are still weak. Note that compared with salted passwords, the ENP proposed in this paper guarantees the diversity of passwords without the need for extra elements (e.g., salt).

Key Stretching: To resist dictionary attack, key stretching , which converts weak passwords to enhanced passwords, was proposed. Key stretching could increase the time cost required to every password attempt, so that the power of defending against dictionary attack is increased. In the ENP proposed in this paper, like key stretching, multi-iteration encryption is used to further improve password security under dictionary attack, and compared with key stretching, the ENP does not introduce extra elements (e.g., salt).

DISADVANTAGES OF EXISTING SYSTEM

- System is not secured due to lack of improved dynamic Key-Hashed Message Authentication Code function (abbreviated as d-HMAC).
- Password protection scheme called Encrypted Negative Password is absent.
- The existing methods (hashed, salted, key stretching) are vulnerable to advanced GPU-based brute force attacks, as they are not designed to handle massive parallel password cracking attempts.
- No support for multi-factor authentication (MFA) or challenge-response mechanisms, leaving the system dependent on only one layer of password security.

PROPOSED SYSTEM

In the proposed system, a password protection scheme called Encrypted Negative Password (abbreviated as ENP) is proposed, which is based on the Negative Database (abbreviated as NDB) [29]–[32], cryptographic hash function and symmetric encryption, and a password authentication framework based on the ENP is presented. The NDB is a new security technique that is inspired by biological immune systems [29] and has a wide range of applications.

- Symmetric encryption is usually deemed inappropriate for password protection. Because the secret key is usually shared by all encrypted passwords and stored together with the authentication data table, once the authentication data table is stolen, the shared key may be stolen at the same time [37]. Thus, these passwords are immediately compromised. However, in the ENP, the secret key is the hash value of the password of each user, so it is almost always different and does not need to be specially generated and stored. Consequently, the ENP enables symmetric encryption to be used for password protection.
- As an implementation of key stretching [38], multi-iteration encryption is introduced to further improve the strength of ENPs. Compared with the salted password scheme and key stretching, the ENP guarantees the diversity of passwords by itself without introducing extra elements (e.g., salt). To summarize, the main contributions of this paper are as follows:
- The system also proposes a password protection scheme called ENP, and we propose two implementations of the ENP: ENPI and ENPII, including their generation algorithms and verification algorithms. Furthermore, a password authentication framework based on the ENP is presented.
- The system analyzes and compares the attack complexity of hashed password, salted password, key stretching and the ENP. The results show that the ENP could resist lookup table attack without the need for extra elements and provide

ADVANTAGES OF PROPOSED SYSTEM:

- The system is more effective due to improved dynamic Key-Hashed Message Authentication Code function (abbreviated as d-HMAC) was proposed for password storage..
- The system's more powerful password scheme by dynamic salt generation and placement are used to improve password security.
- The use of Encrypted Negative Password (ENP) eliminates the need for storing extra elements like salt or shared keys, thereby reducing risks related to key leakage or database theft.
- The multi-iteration encryption process increases resistance to brute-force and dictionary attacks, improving overall system robustness without sacrificing performance.

CHAPTER – 4

SYSTEM REQUIREMENTS

4.1 FUNCTIONAL REQUIREMENTS

1. DATA OWNER
2. DATA USER
3. WEB SERVER
4. ATTACKER

4.2 NON-FUNCTIONAL REQUIREMENTS

4.2.1 HARDWARE REQUIREMENTS

MINIMUM (Required for Execution)		MY SYSTEM (Development)
System	Pentium IV 2.2 GHz	i3 Processor 5th Gen
Hard Disk	20 Gb	512 Gb
Ram	1 Gb	4 Gb

4.2.2 SOFTWARE REQUIREMENTS

Operating System	Windows 10/11
Development Software	JAVA 8.1
Programming Language	Java
Front End Technologies	HTML5, CSS3, Java Script
Database Language	SQL
Database (RDBMS)	MySQL
Database Software	MySQL Server
Web Server or Deployment Server	Apache tomcat
Design/Modelling	Rational Rose

CHAPTER–5

SYSTEM STUDY

5.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

5.2 FEASIBILITY ANALYSIS

A feasibility study assesses the operational, technical and economic metrics of the proposed project. The feasibility study is intended to be a preliminary review of the facts to see if it is worthy of proceeding to the analysis phase. From the system analyst perspective, the feasibility analysis is the primary tool for recommending whether to proceed to the next phase or to discontinue the project.

Three key considerations involved in the feasibility analysis are

- SOCIAL FEASIBILITY
- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

ECONOMICAL FEASIBILITY

In this we carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

In this we carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system

CHAPTER –6

SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE

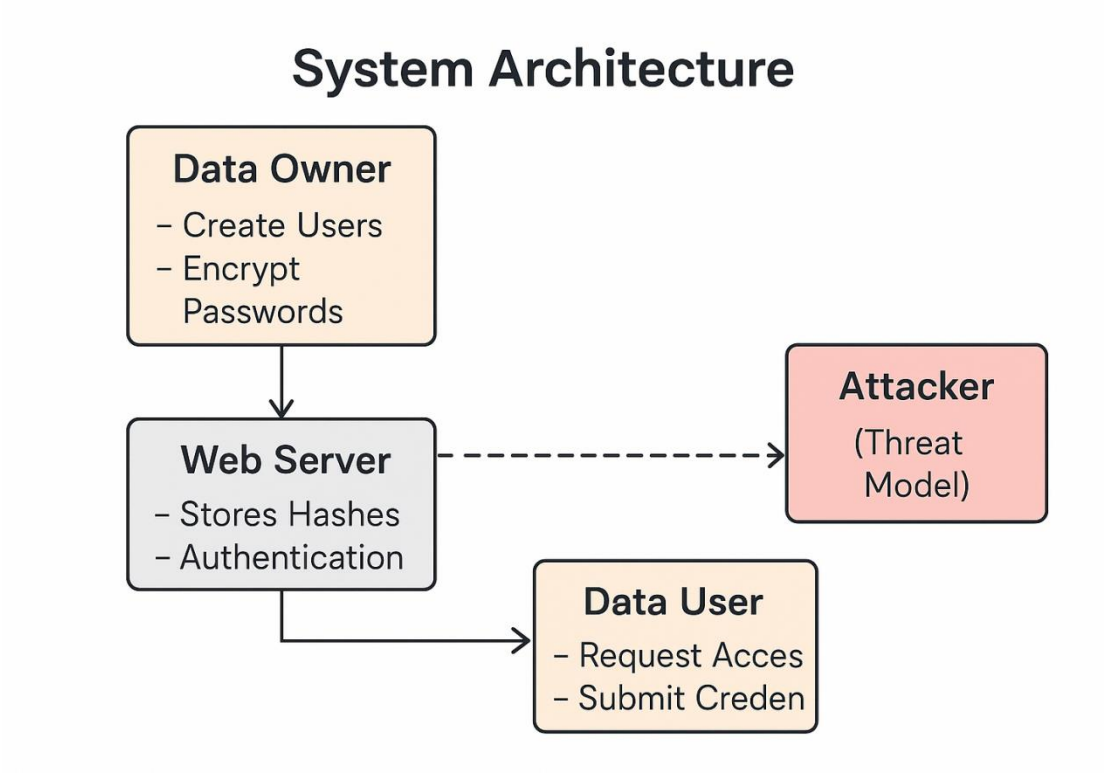


Fig- 6.1 System Architecture

6.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

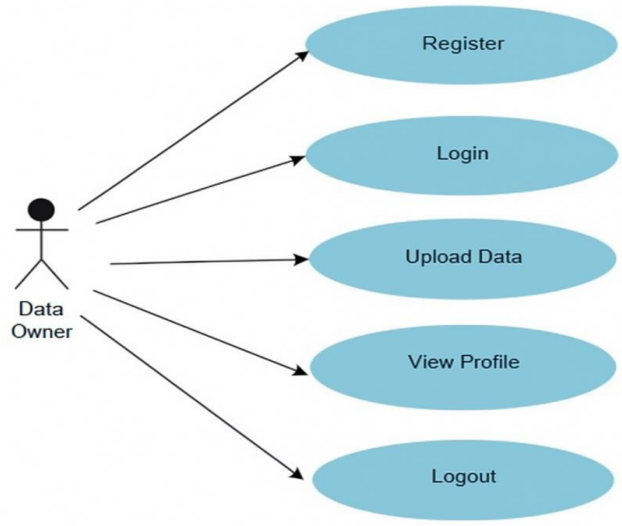
GOALS:

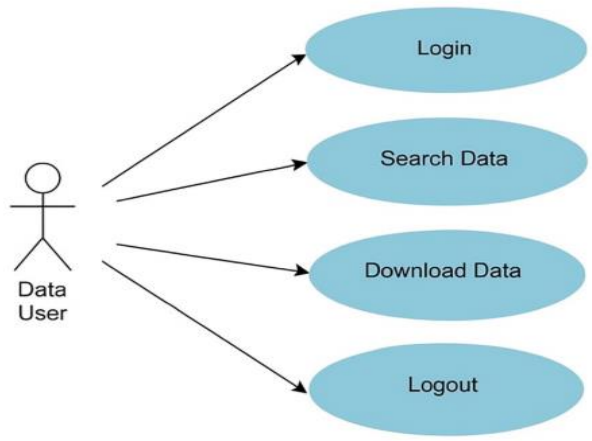
The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extensibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.

6.2.1 USECASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted





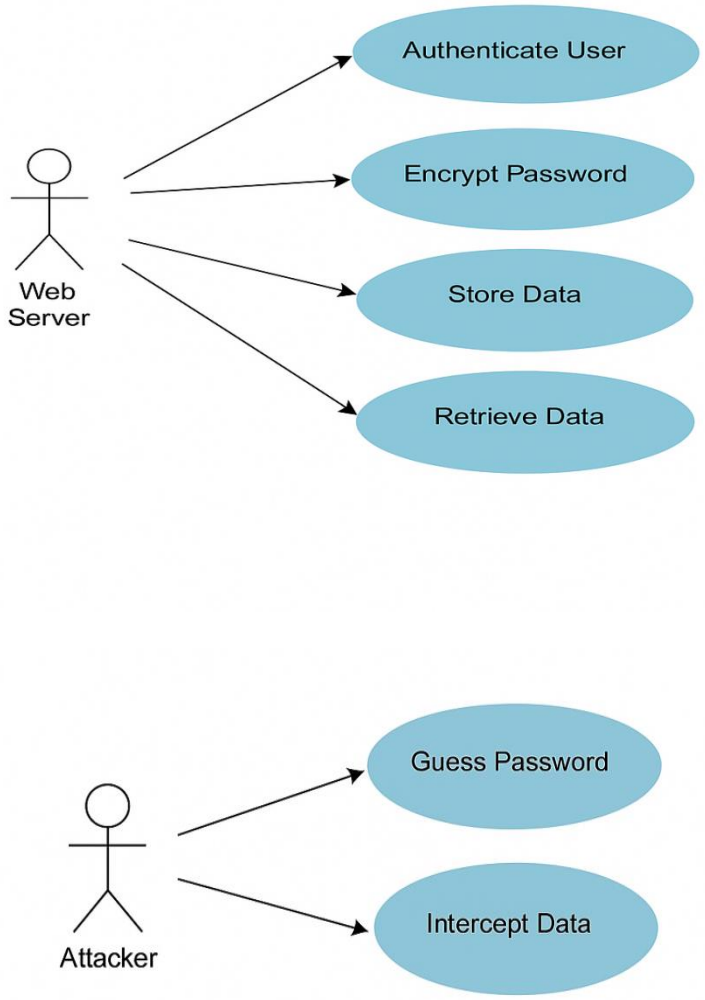


Fig- 6.2 UML Diagram

6.2.2 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

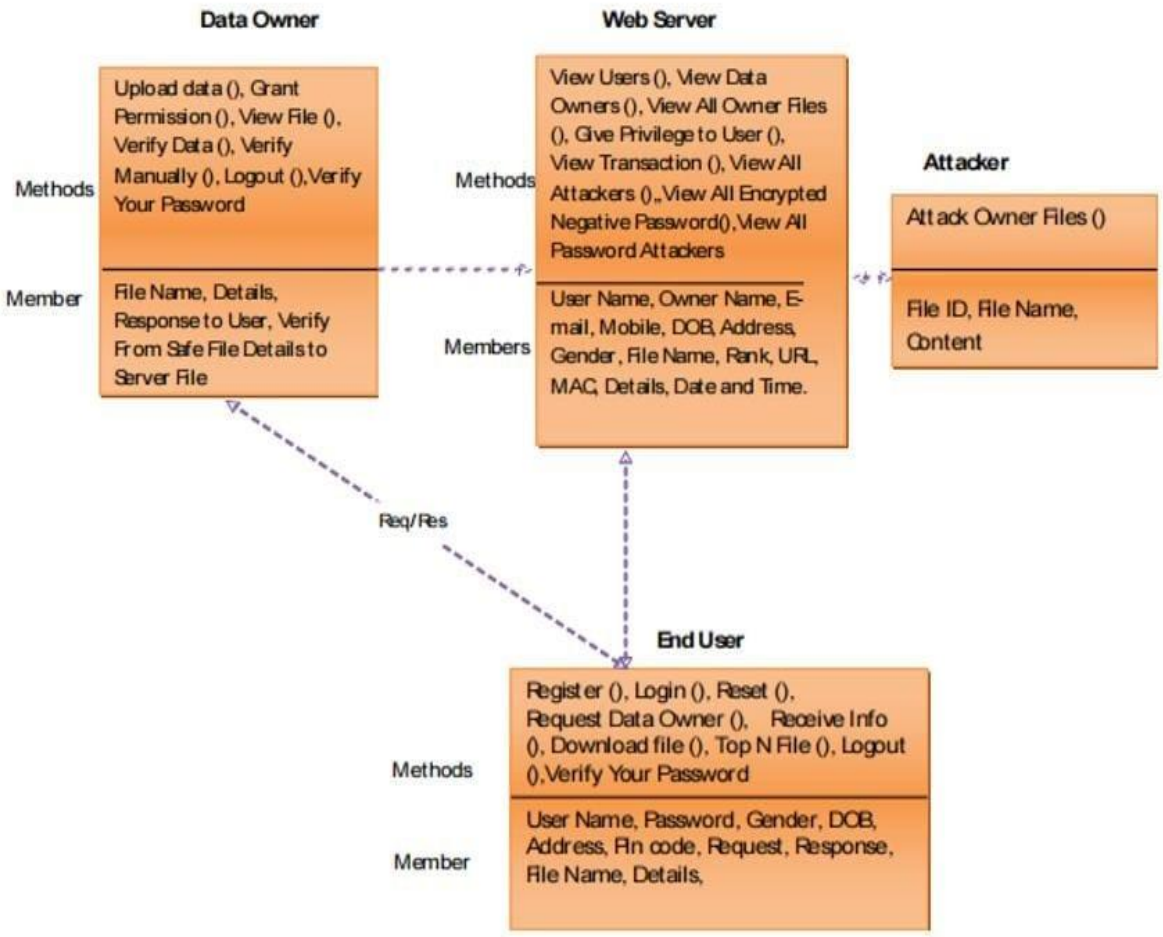


Fig- 6.2.2 Class Diagram

6.2.3 SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

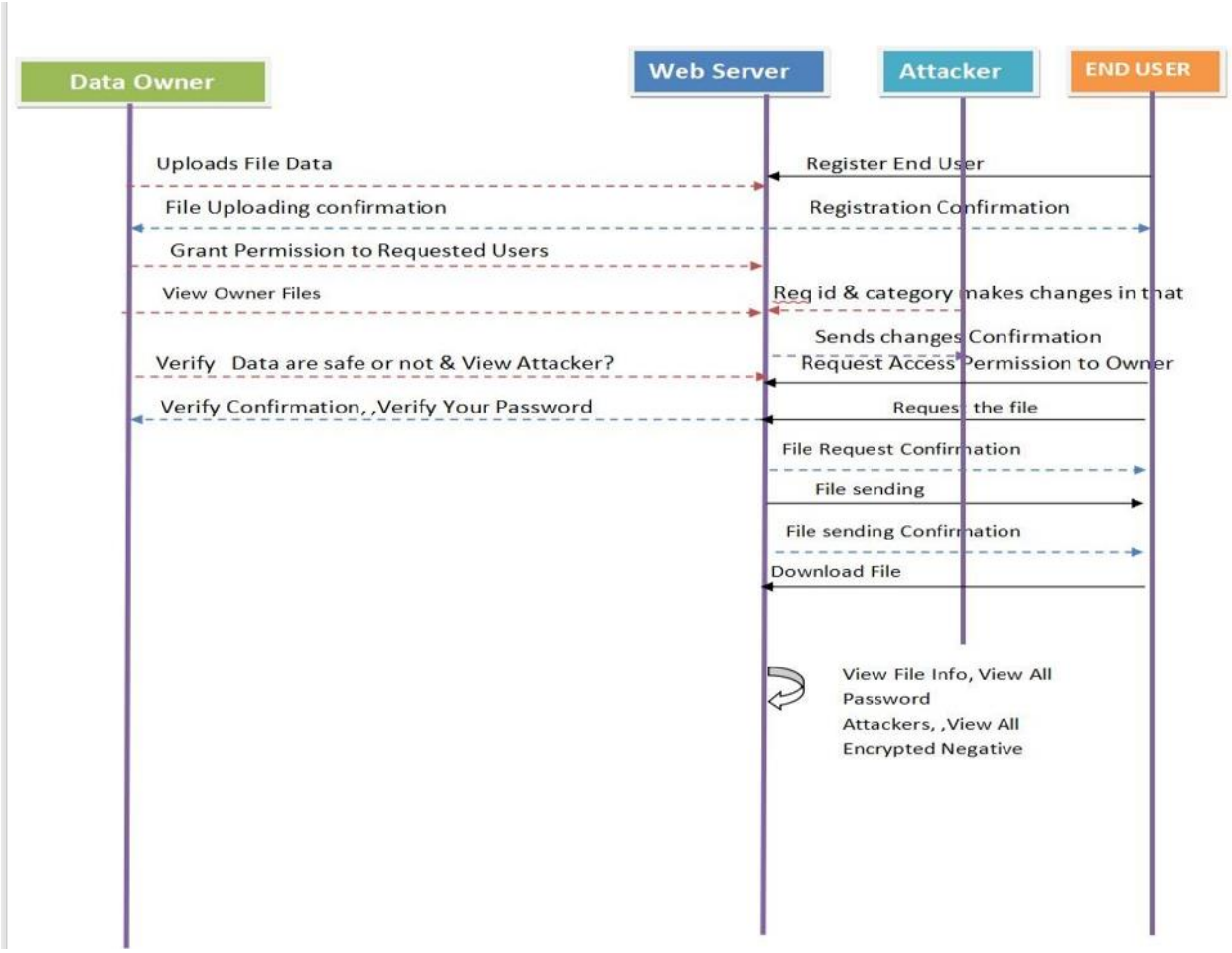


Fig- 6.2.3 Sequence Diagram

6.2.4 COLLABORATION DIAGRAM

In UML diagrams, collaboration is a type of structured classifier in which roles and attributes co-operate to define the internal structure of a classifier. You use collaboration when you want to define only the roles and connections that are required to accomplish a specific goal of the collaboration.

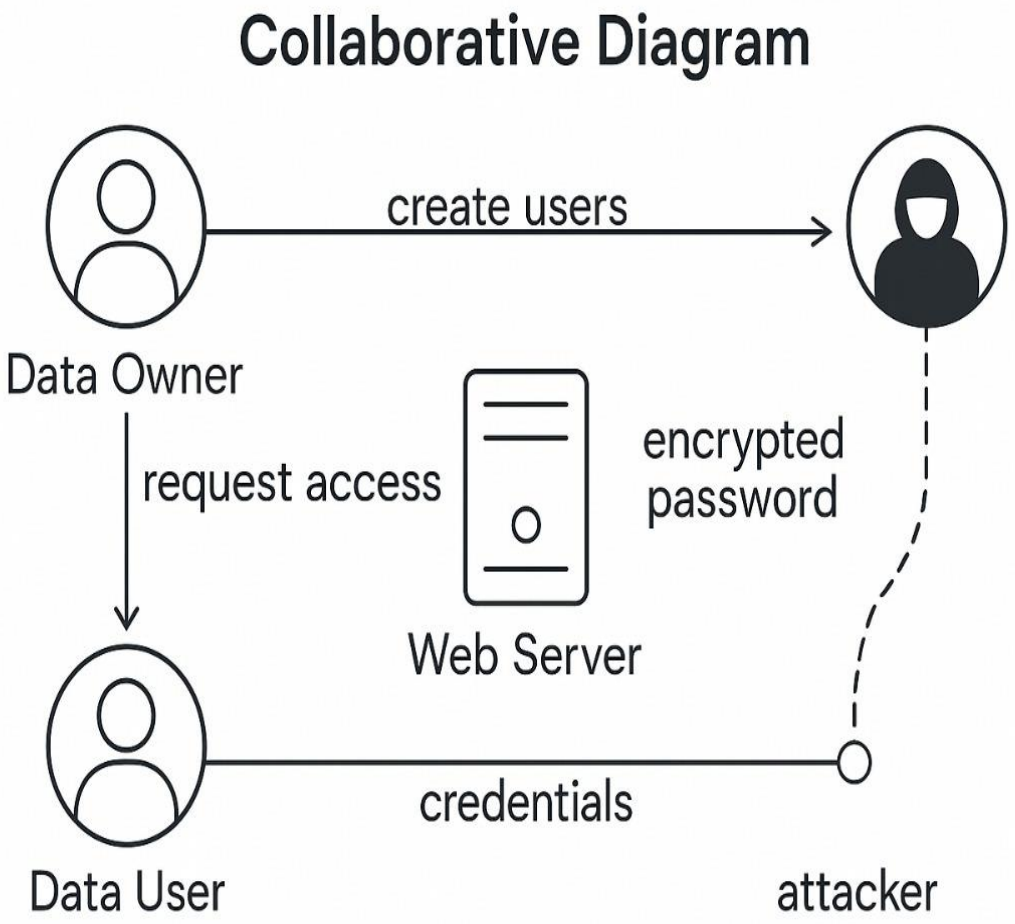


Fig- 6.2.4 Collaboration Diagram

6.2.5 ACTIVITY DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control

.

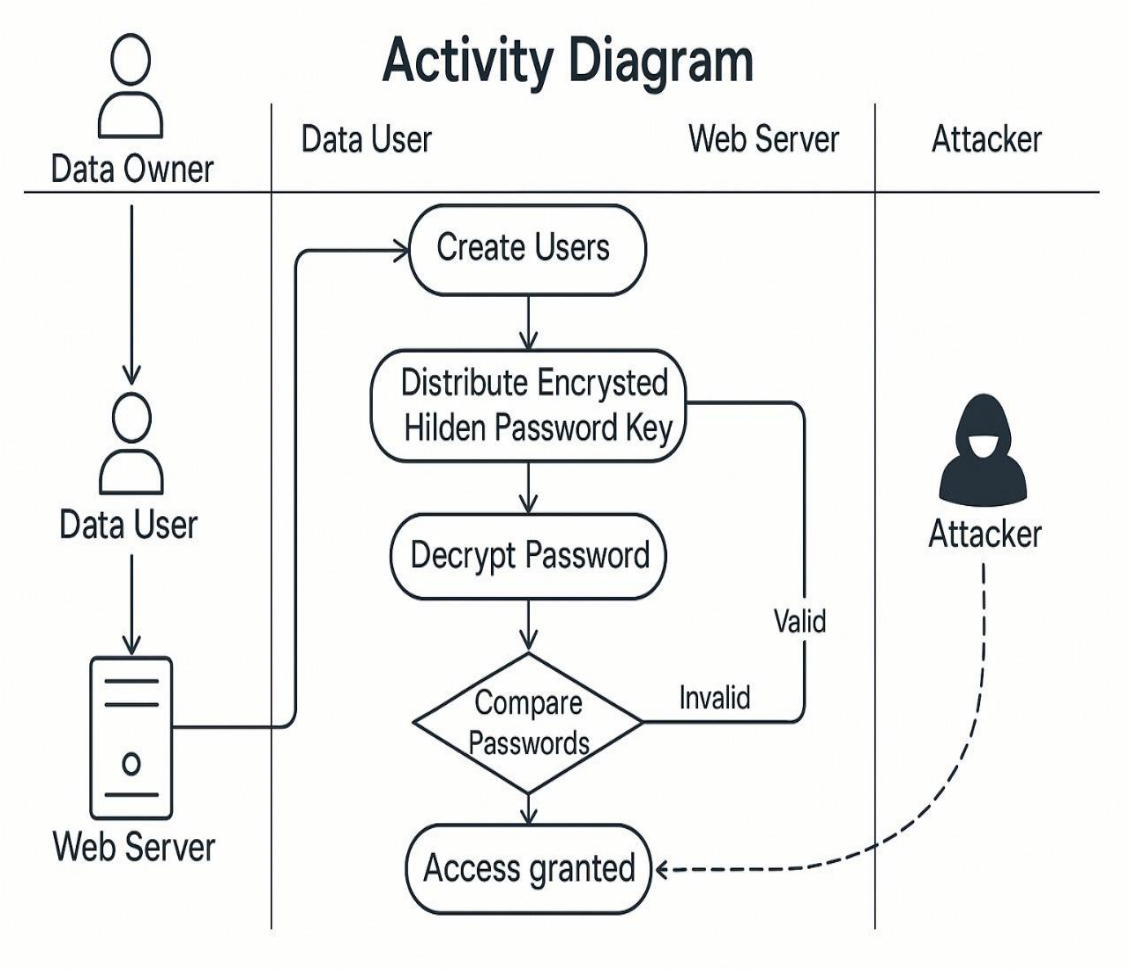


Fig- 6.2.5 Activity Diagram

6.2.6 COMPONENT DIAGRAM

A component diagram is used to break down a large object-oriented system into the smaller components, so as to make them more manageable. It models the physical view of a system such as executables, files, libraries, etc. that resides within the node It visualizes the relationships as well as the organization between the components present in the system. It helps in forming an executable system. A component is a single unit of the system, which is replaceable and executable. The implementation details of a component are hidden, and it necessitates an interface to execute a function. It is like a black box whose behavior is explained by the provided and required interfaces.

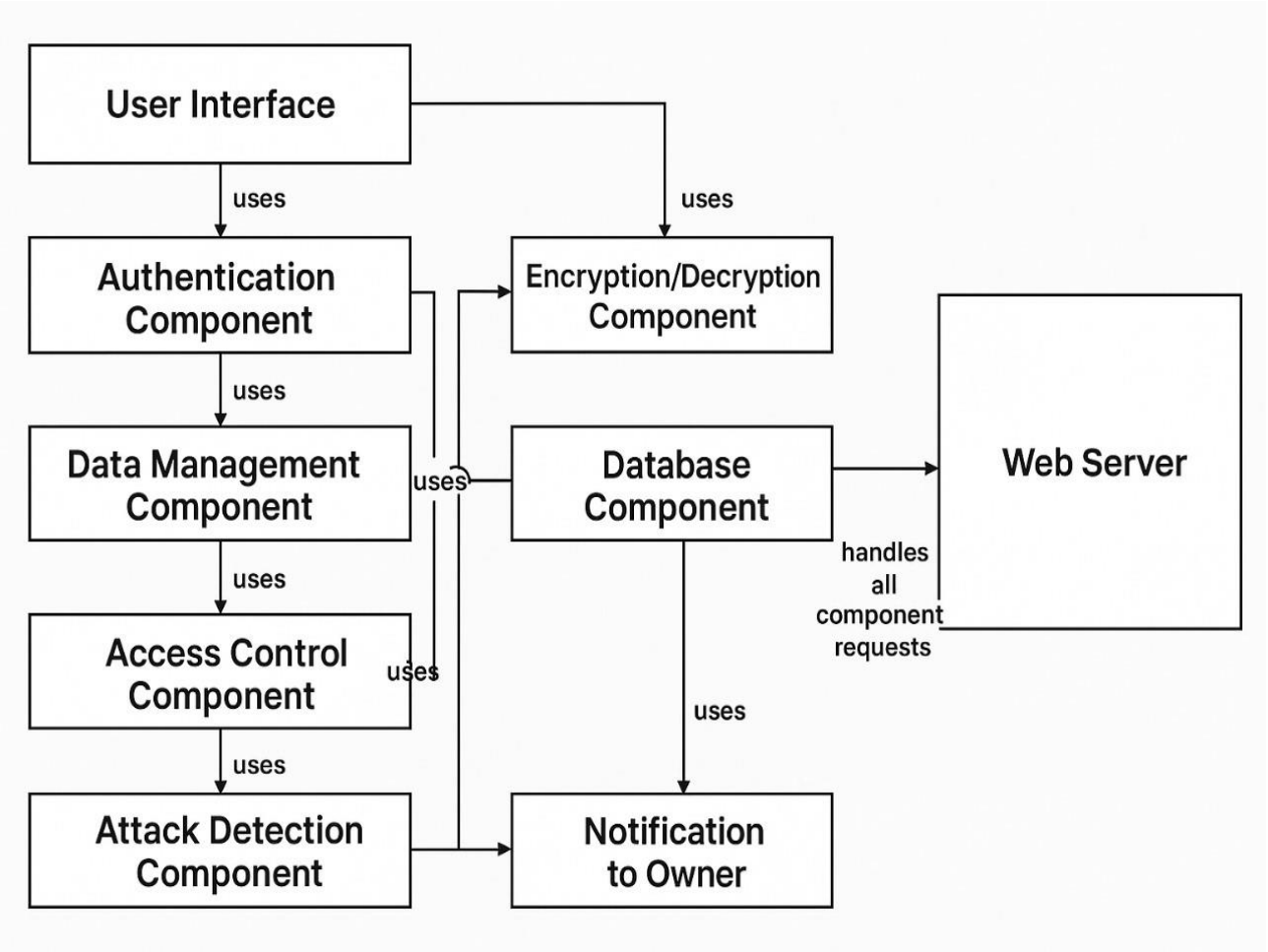


Fig- 6.2.6 Component Diagram

6.2.7 DEPLOYMENT DIAGRAM

A deployment diagram is a UML diagram type that shows the execution architecture of a system, including nodes such as hardware or software execution environments, and the middleware connecting them.

Deployment diagrams are typically used to visualize the physical hardware and software of a system. Using it you can understand how the system will be physically deployed on the hardware. Deployment diagrams help model the hardware topology of a system compared to other UML diagram types which mostly outline the logical components of a system.

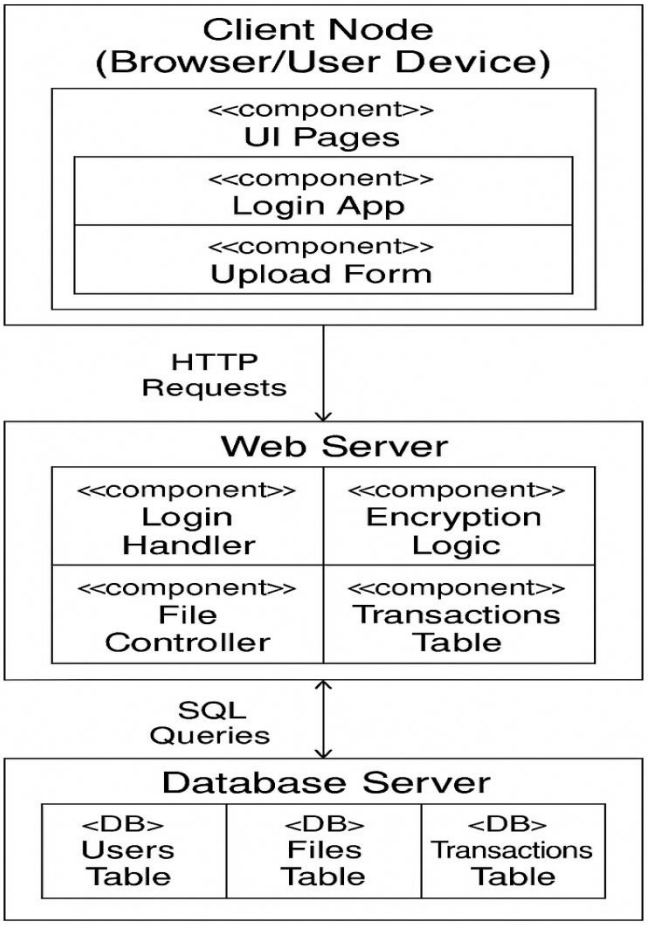


Fig- 6.2.7 Deployment Diagram

6.2.8 ER DIAGRAM

The Entity Relational Model is a model for identifying entities to be represented in the database and representation of how those entities are related. The ER data model specifies enterprise schema that represents the overall logical structure of a database graphically.

The Entity Relationship Diagram explains the relationship among the entities present in the database. ER models are used to model real-world objects like a person, a car, or a company and the relation between these real-world objects. In short, the ER Diagram is the structural format of the database.

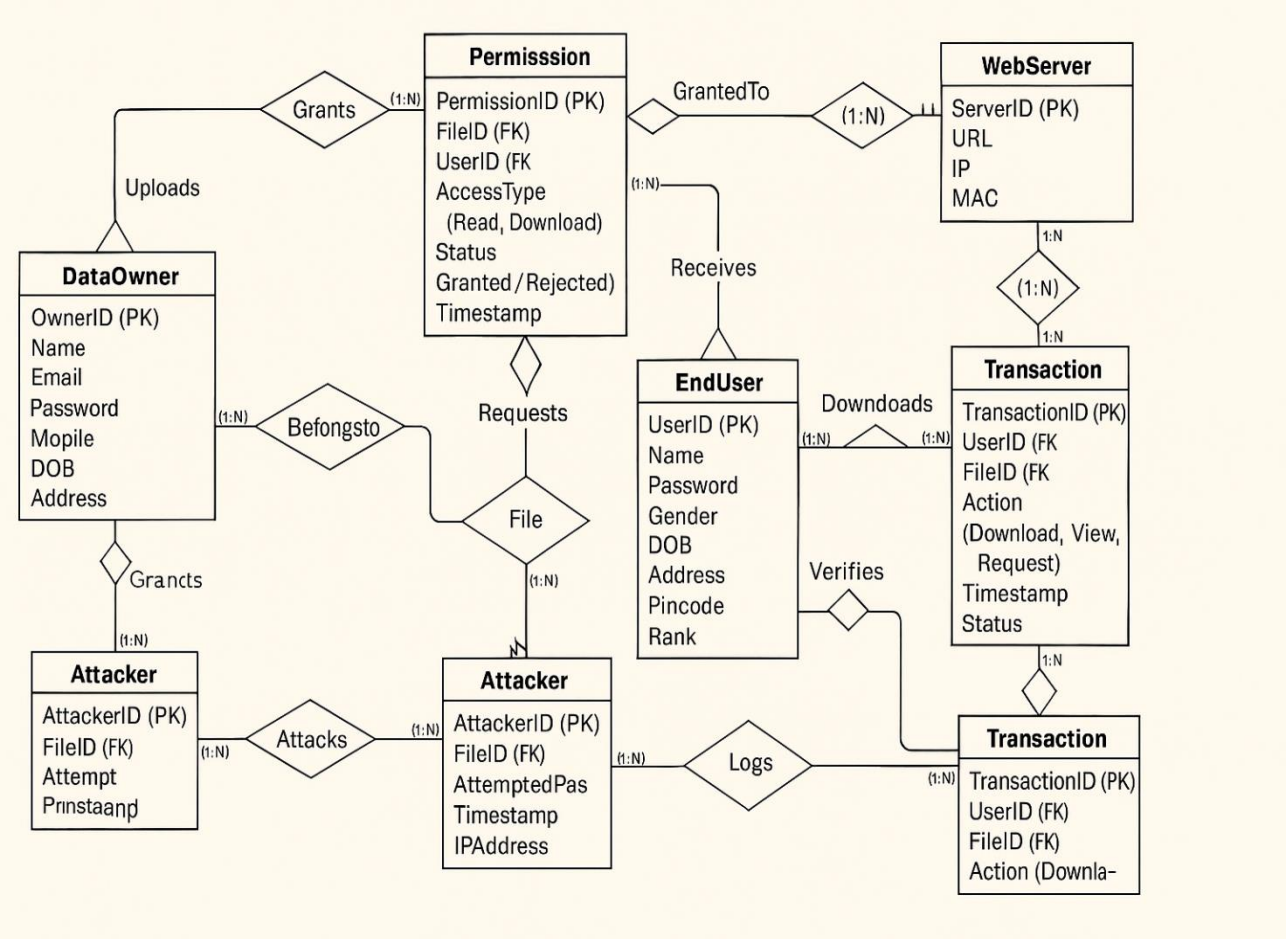


Fig- 6.2.8 ER Diagram

CHAPTER – 7

INPUT OUTPUT DESIGN

7.1 INPUT DESIGN

Input Design plays a vital role in the life cycle of software development; it requires very careful attention of developers. The input design is to feed data to the application as accurate as possible. So, inputs are supposed to be designed effectively so that the errors occurring while feeding are minimized. According to Software Engineering Concepts, the input forms or screens are designed to provide to have a validation control over the input limit, range and other related validations.

Input design is the process of converting the user created input into a computer-based format. The goal of the input design is to make the data entry logical and free from errors. The error is in the input are controlled by the input design. The application has been developed in user-friendly manner. The forms have been designed in such a way during the processing the cursor is placed in the position where must be entered. The user is also provided with in an option to select an appropriate input from various alternatives related to the field in certain cases.

7.2 OUTPUT DESIGN

The Output from the computer is required to mainly create an efficient method of communication within the company primarily among the project leader and his team members, in other words, the administrator and the clients. The output of VPN is the system which allows the project leader to manage his clients in terms of creating new clients and assigning new projects to them, maintaining a record of the project validity and providing folder level access to each client on the user side depending on the projects allotted to him. User authentication procedures are maintained at the initial stages itself.

The application starts running when it is executed for the first time. The server has to be started and then the internet explorer in used as the browser. The project will run on the local area network so the server machine will serve as the administrator while the other connected systems can act as the clients. The developed system is highly user friendly and can be easily understood by anyone using it even for the first time

CHAPTER – 8

IMPLEMENTATION

MODULES

1. DATA OWNER
2. DATA USER
3. WEB SERVER
4. ATTACKER

MODULE DESCRIPTION

1. DATA OWNER

The data owner uploads sensitive data with encrypted passwords to ensure secure access.They define access rules and manage who can retrieve or modify the encrypted content.

2. WEB SERVER

The web server stores encrypted data and handles access requests securely. It ensures only authenticated users can retrieve or decrypt the protected data.

3. DATA USER

The data consumer accesses the encrypted data using secure login credentials. They decrypt data locally after successful authentication using hidden password mechanisms.

4. ATTACKER

The attacker tries to breach the system to steal or tamper with encrypted passwords. Security measures like hidden encryption and access control prevent such unauthorized access.

CHAPTER – 9

SOFTWARE ENVIRONMENT

Java Technology:

Java technology is both a programming language and a platform.

The Java Programming Language:

The Java programming language is a high-level language that can be characterized by all of the following buzzwords:

- Simple
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Java programming language is unusual in that a program is both compiled and interpreted. With the compiler, first you translate a program into an intermediate language called *Java byte codes* —the platform-independent codes interpreted by the interpreter on the Java platform. The interpreter parses and runs each Java byte code instruction on the computer. Compilation happens just once; interpretation occurs each time the program is executed. The following figure illustrates how this works.

You can think of Java byte codes as the machine code instructions for the *Java Virtual Machine* (Java VM). Every Java interpreter, whether it's a development tool or a Web browser that can run applets, is an implementation of the Java VM. Java byte codes help make "write once, run anywhere" possible. You can compile your program into byte codes on any platform that has a Java compiler. The byte codes can then be run on any implementation of the Java VM.

That means that as long as computer has a Java VM, the same program written in the Java programming language can run on Windows 2000, a Solaris workstation, or on an iMac.

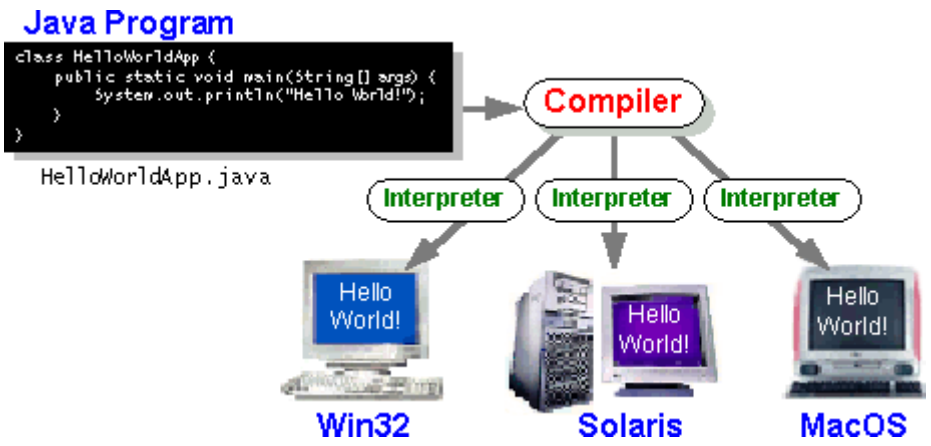


Fig- 9.2.1 Java Program Language

The Java Platform

A *platform* is the hardware or software environment in which a program runs. We've already mentioned some of the most popular platforms like Windows 2000, Linux, Solaris, and MacOS. Most platforms can be described as a combination of the operating system and hardware. The Java platform differs from most other platforms in that it's a software-only platform that runs on top of other hardware- based platforms.

The Java platform has two components:

- The *Java Virtual Machine* (Java VM)
- The *Java Application Programming Interface* (Java API)

The Java API is a large collection of ready-made software components that provide many useful capabilities, such as graphical user interface (GUI) widgets. The Java API is grouped into libraries of related classes and interfaces; these libraries are known as *packages*. The next section, What Can Java Technology Do? Highlights what functionality some of the packages in the Java API provide.

What Can Java Technology Do?

The most common types of programs written in the Java programming language are *applets* and *applications*. If you've surfed the Web, you're probably already familiar with applets. An applet is a program that adheres to certain conventions that allow it to run within a Java-enabled browser. An application is a standalone program that runs directly on the Java

platform. A special kind of application known as a *server* serves and supports clients on a network. Examples of servers are Web servers, proxy servers, mail servers, and print servers. Another specialized program is a *servlet*. A servlet can almost be thought of as an applet that runs on the server side. Java Servlets are a popular choice for building interactive web applications, replacing the use of CGI scripts. Servlets are similar to applets in that they are runtime extensions of applications. Instead of working in browsers, though, servlets run within Java Web servers, configuring or tailoring the server.

How does the API support all these kinds of programs? It does so with packages of software components that provides a wide range of functionality. Every full implementation of the Java platform gives you the following features:

- **The essentials**: Objects, strings, threads, numbers, input and output, data structures, system properties, date and time, and so on.
- **Applets**: The set of conventions used by applets.
- **Networking**: URLs, TCP (Transmission Control Protocol), UDP (User Data gram Protocol) sockets, and IP (Internet Protocol) addresses.
- **Internationalization**: Help for writing programs that can be localized for users worldwide. Programs can automatically adapt to specific locales and be displayed in the appropriate language.
- **Security**: Both low level and high level, including electronic signatures, public and private key management, access control, and certificates.
- **Software components**: Known as JavaBeans™, can plug into existing component architectures.
- **Object serialization**: Allows lightweight persistence and communication via Remote Method Invocation (RMI). **Java Database Connectivity (JDBC™)**: Provides uniform access to a wide range of relational databases.

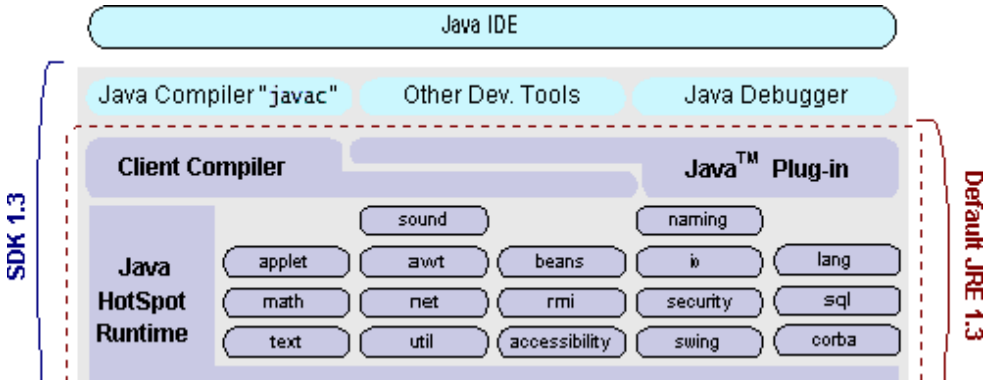


Fig - 9.2.2 Java IDE

How Will Java Technology Change My Life?

We can't promise you fame, fortune, or even a job if you learn the Java programming language. Still, it is likely to make your programs better and requires less effort than other languages. We believe that Java technology will help you do the following:

- **Get started quickly**: Although the Java programming language is a powerful object- oriented language, it's easy to learn, especially for programmers already familiar with C or C++.
- **Write less code**: Comparisons of program metrics (class counts, method counts, and so on) suggest that a program written in the Java programming language can be four times smaller than the same program in C++.
- **Write better code**: The Java programming language encourages good coding practices, and its garbage collection helps you avoid memory leaks. Its object orientation, its JavaBeans component architecture, and its wide-ranging, easily extendible API let you reuse other people's tested code and introduce fewer bugs.
- **Develop programs more quickly**: Your development time may be as much as twice as fast versus writing the same program in C++. Why? You write fewer lines of code and it is a simpler programming language than C++.
- **Avoid platform dependencies with 100% Pure Java**: You can keep your program portable by avoiding the use of libraries written in other languages. The 100% Pure Java™ Product Certification Program has a repository of historical process manuals, white papers, brochures, and similar materials online.
- **Write once, run anywhere**: Because 100% Pure Java programs are compiled into machine-independent byte codes, they run consistently on any Java

platform.

- **Distribute software more easily**: You can upgrade applets easily from a central server. Applets take advantage of the feature of allowing new classes to be loaded "on the fly," without recompiling the entire program.

ODBC

Microsoft Open Database Connectivity (ODBC) is a standard programming interface for application developers and database systems providers. Before ODBC became a *de facto* standard for Windows programs to interface with database systems, programmers had to use proprietary languages for each database they wanted to connect to. Now, ODBC has made the choice of the database system almost irrelevant from a coding perspective, which is as it should be. Application developers have much more important things to worry about than the syntax that is needed to port their program from one database to another when business needs suddenly change.

Through the ODBC Administrator in Control Panel, you can specify the particular database that is associated with a data source that an ODBC application program is written to use. Think of an ODBC data source as a door with a name on it. Each door will lead you to a particular database. For example, the data source named Sales Figures might be a SQL Server database, whereas the Accounts Payable data source could refer to an Access database. The physical database referred to by a data source can reside anywhere on the LAN. The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as SQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a stand-alone program called ODBCADM.EXE. There is a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC data sources.

From a programming perspective, the beauty of ODBC is that the application can be written to use the same set of function calls to interface with any data source, regardless of the database vendor. The source code of the application doesn't change whether it talks to Oracle or SQL Server. We only mention these two as an example. There are ODBC drivers available for several dozen popular database systems. The loading of the ODBC drivers is transparent to the ODBC application program. In a client/server environment, the ODBC API even handles many of the network issues for the application programmer. Microsoft has always claimed that the critical factor in performance is the quality of the driver software that is used. In our humble opinion, this is true. The availability of good ODBC drivers has improved a great deal recently.

And anyway, the criticism about performance is somewhat analogous to those who said that compilers would never match the speed of pure assembly language. Maybe not, but the compiler (or ODBC) gives you the opportunity to write cleaner programs, which means you finish sooner. Meanwhile, computers get faster every year.

JDBC

In an effort to set an independent database standard API for Java; Sun Microsystems developed Java Database Connectivity, or JDBC. JDBC offers a generic SQL database access mechanism that provides a consistent interface to a variety of RDBMSs. This consistent interface is achieved through the use of "plug-in" database connectivity modules, or *drivers*. If a database vendor wishes to have JDBC support, he or she must provide the driver for each platform that the database and Java run on. To gain a wider acceptance of JDBC, Sun based JDBC's framework on ODBC. As you discovered earlier in this chapter, ODBC has widespread support on a variety of platforms , Because of user input, the final JDBC v1.0 specification was released soon after. The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book

JDBC Goals

Few software packages are designed without goals in mind. JDBC is one that, because of its many goals, drove the development of the API. These goals, in conjunction with early reviewer feedback, have finalized the JDBC class library into a solid framework for building database applications in Java. The goals that were set for JDBC are important. They will give you some insight as to why certain classes and functionalities behave the way they do. The eight design goals for JDBC are as follows:

1. **SQL Level API**

The designers felt that their main goal was to define a SQL interface for Java. Although not the lowest database interface level possible, it is at a low enough level for higher-level tools and APIs to be created. Conversely, it is at a high enough level for application programmers to use it confidently. Attaining this goal allows for future tool vendors to "generate" JDBC code and to hide many of JDBC's complexities from the end user.

2. **SQL Conformance**

SQL syntax varies as you move from database vendor to database vendor. In an effort to support a wide variety of vendors, JDBC will allow any query statement to be passed through it to the underlying database driver. This allows the connectivity module to handle non-standard functionality in a manner that is suitable for its users.

3. **JDBC must be implemental on top of common database interfaces**

The JDBC SQL API must "sit" on top of other common SQL level APIs. This goal allows JDBC to use existing ODBC level drivers by the use of a software interface. This interface would translate JDBC calls to ODBC and vice versa.

4. **Use strong, static typing wherever possible**

Strong typing allows for more error checking to be done at compile time; also, less error appear at runtime.

5. **Keep the common cases simple**

Because more often than not, the usual SQL calls used by the programmer are simple SELECT's, INSERT's, DELETE's and UPDATE's, these queries should be simple to perform with JDBC.

Networking

TCP/IP stack

The TCP/IP stack is shorter than the OSI one:

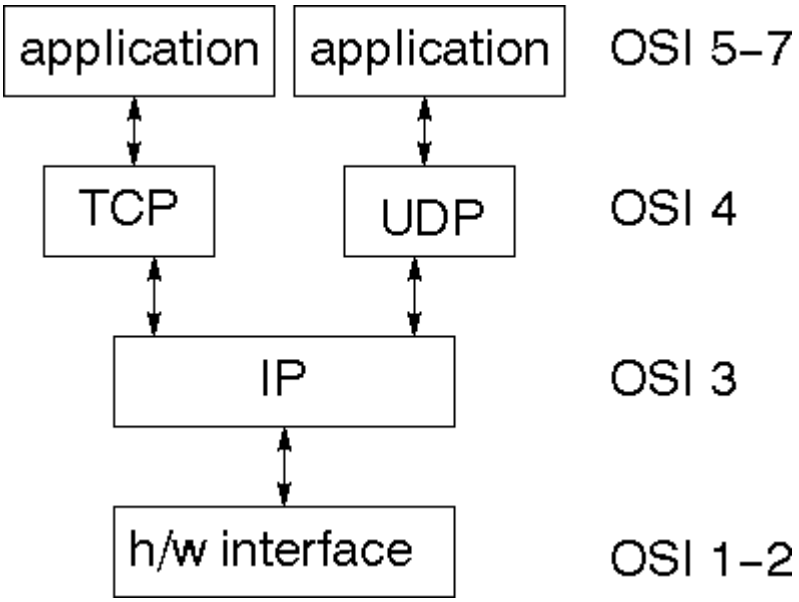


Fig-9.2.3 TCP/IP stack

TCP is a connection-oriented protocol; UDP (User Datagram Protocol) is a connectionless protocol.

IP datagram's

The IP layer provides a connectionless and unreliable delivery system. It considers each datagram independently of the others. Any association between datagram must be supplied by the higher layers. The IP layer supplies a checksum that includes its own header. The header includes the source and destination addresses. The IP layer handles routing through an Internet. It is also responsible for breaking up large datagram into smaller ones for transmission and reassembling them at the other end.

UDP

UDP is also connectionless and unreliable. What it adds to IP is a checksum for the contents of the datagram and port numbers. These are used to give a client/server model - see later.

TCP

TCP supplies logic to give a reliable connection-oriented protocol above IP. It provides a virtual circuit that two processes can use to communicate.

Internet addresses

In order to use a service, you must be able to find it. The Internet uses an address scheme for machines so that they can be located. The address is a 32 bit integer which gives the IP address. This encodes a network ID and more addressing. The network ID falls into various classes according to the size of the network address.

Network address

Class A uses 8 bits for the network address with 24 bits left over for other addressing. Class B uses 16-bit network addressing. Class C uses 24 bit network addressing and class D uses all 32.

Subnet address

Internally, the UNIX network is divided into sub networks. Building 11 is currently on one sub network and uses 10-bit addressing, allowing 1024 different hosts.

Host address

8 bits are finally used for host addresses within our subnet. This places a limit of 256 machines that can be on the subnet.

Port addresses

A service exists on a host, and is identified by its port. This is a 16 bit number. To send a message to a server, you send it to the port for that service of the host that it is running on. This is not location transparency! Certain of these ports are "well known".

JFree Chart

JFreeChart is a free 100% Java chart library that makes it easy for developers to display professional quality charts in their applications. JFreeChart's extensive feature set includes:A consistent and well-documented API, supporting a wide range of chart types;

1. Map Visualizations

Charts showing values that relate to geographical areas. Some examples include: (a) population density in each state of the United States, (b) income per capita for each country in Europe, (c) life expectancy in each country of the world. The tasks in this project include:Sourcing freely redistributable vector outlines for the countries of the world, states/provinces in particular countries (USA in particular, but also other areas); Creating an appropriate dataset interface (plus default implementation), a rendered, and integrating this with the existing XYPlot class in JFreeChart; Testing, documenting, testing some more, documenting some more.

2. Time Series Chart Interactivity

Implement a new (to JFreeChart) feature for interactive time series charts --- to display a separate control that shows a small version of ALL the time series data, with a sliding "view" rectangle that allows you to select the subset of the time series data to display in the main chart.

3. Dashboards

There is currently a lot of interest in dashboard displays. Create a flexible dashboard mechanism that supports a subset of JFreeChart chart types (dials, pies, thermometers, bars, and lines/time series) that can be delivered easily via both Java Web Start and an applet.

4. Property Editors

The property editor mechanism in JFreeChart only handles a small subset of the properties that can be set for charts. Extend (or reimplement) this mechanism to provide greater end-user control over the appearance of the charts.

Tomcat 6.0 web server

Tomcat is an open-source web server developed by Apache Group. Apache Tomcat is the servlet container that is used in the official Reference Implementation for the Java Servlet and JavaServer Pages technologies. The Java Servlet and Java Server Pages specifications are developed by Sun under the Java Community Process. Web Servers like Apache Tomcat support only web components while an application server supports web components as well as business components (BEAs WebLogic, is one of the popular application server).To develop a web application with jsp/servlet install any web server like Run, Tomcat etc. toyou're your application.

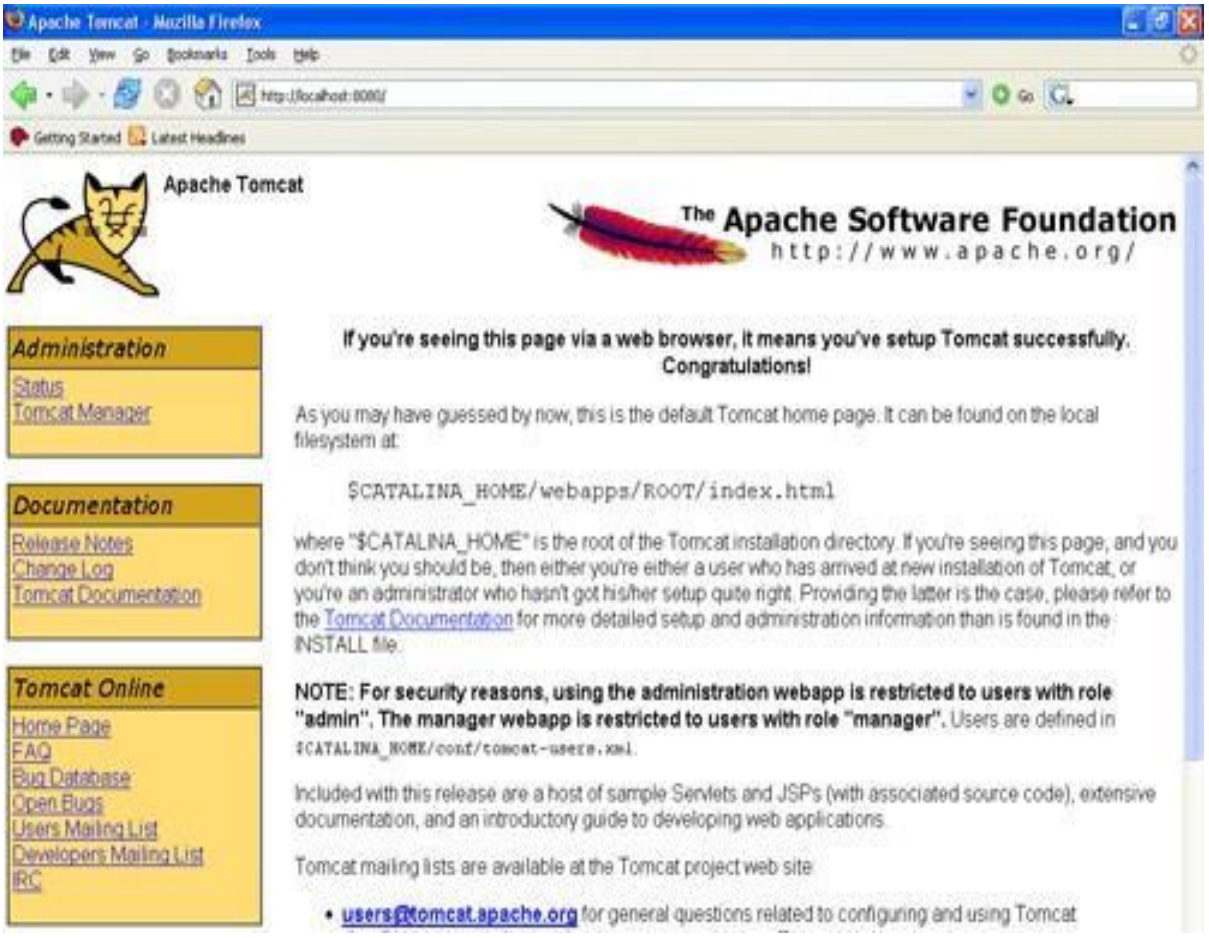


Fig-9.2.4 Tomcat Webserver

9.2 Source code

WebserverMain.jsp

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Web Server Main Page</title>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<link href="style.css" rel="stylesheet" type="text/css" />
<script type="text/javascript" src="js/jquery.js"></script>
<script type="text/javascript" src="js/cufon-yui.js"></script>
<script type="text/javascript" src="js/arial.js"></script>
<script type="text/javascript" src="js/cuf_run.js"></script>
<script type="text/javascript" src="js/radius.js"></script>
<style type="text/css">
<!--
.style1 {color: #1f7fbb}
.style2 {color: #FF0000}
.style3 {     color: #FF0000;
      font-weight: bold;
}
-->
</style>
</head>
<body>
<div class="main">
  <div class="header">
    <div class="header_resize">
      <div class="logo">
        <h2><a href="index.html">SECURE ACCESS WITH HIDDEN PASSWORD
ENCRYPTION</a></h2>
      </div>
      <div class="clr"></div>
      <div class="menu_nav">
        <ul>
          <li><a href="index.html">Home</a></li>
          <li><a href="endUserLogin.html">Client</a></li>
          <li><a href="dataOwnerLogin.html"></a></li>
          <li class="active"><a href="webServerLogin.html">WebServer</a></li>
          <li></li>
                <li></li>
        </ul>
      </div>
      <div class="clr"></div>
      <img src="images/main_img.jpg" alt="" width="972" height="313" style="padding:25px 0;"
/>
      <div class="clr"></div>
    </div>
```

```
    </div>
    <div class="clr"></div>
    <div class="content">
      <div class="content_resize">
        <div class="mainbar">
          <div class="article">
            <h2><span>WelCome To <span
class="style1"><%=(String)application.getAttribute("server")%> </span>Main...!</span></h2>
            <div class="clr"></div>

            <img src="images/img_2.jpg" width="613" height="193" alt="" />
            <div class="clr"></div>
            <p></p>

            <div align="justify" class="style2"><span class="style3">In this paper, we propose a
password authentication framework that is designed for secure password storage and could be
easily integrated into existing authentication systems. In our framework, first, the received plain
password from a client is hashed through a cryptographic hash function (e.g., SHA-256). Then,
the hashed password is converted into a negative password. Finally, the negative password is
encrypted into an Encrypted Negative Password (abbreviated as ENP) using a symmetric-key
algorithm (e.g., AES), and multi-iteration encryption could be employed to further improve
security. The cryptographic hash function and symmetric encryption make it difficult to crack
passwords from ENPs.</span></div>
          </div>
        </div>
        <div class="sidebar">
          <div class="gadget">
            <h2>Sidebar Menu</h2>
            <div class="clr"></div>
            <ul class="sb_menu">
              <li><a href="viewAllEndUsers.jsp?ab">View All Users</a></li>

              <li><a href="viewAllDataOwnersFiles.jsp">View All Owner Files</a></li>
              <li><a href="serverPermission.jsp">Give Privileges To Users </a></li>
                          <li><a href="viewAllTransaction.jsp">View Transactions</a></li>
                          <li><a href="S_viewAllENPSetails.jsp">View All Encrypted Negative
Password</a></li>
              <li><a href="attackers.jsp">View All Attacker</a></li>
                          <li><a href="pattackers.jsp">View All Password Attackers</a></li>
                          <li><a href="S_view_File_Score.jsp">View File Score Results</a></li>

                          <li><a href="webServerLogin.html">Log Out</a></li>
            </ul>
          </div>
        </div>
        <div class="clr"></div>
        <div class="fbg"></div>
        <div class="clr"></div>
      </div>
    </div>
    <div class="footer">
```

```
      <div class="clr"></div>
    </div>
  </div>
  </body>
  </html>
```

dataOwner.jsp

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

<title>Data Owner Main Page</title>

<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

<link href="style.css" rel="stylesheet" type="text/css" />

<script type="text/javascript" src="js/jquery.js"></script>

<script type="text/javascript" src="js/cufon-yui.js"></script>

<script type="text/javascript" src="js/arial.js"></script>

<script type="text/javascript" src="js/cuf_run.js"></script>

<script type="text/javascript" src="js/radius.js"></script>

<style type="text/css">

<!--

.style1 {color: #1f7fbb}

.style2 {color: #FF0000}

.style3 {	color: #FF0000;

	font-weight: bold;

}

-->

</style>

</head>

<body>

<div class="main">
```

```
  <div class="header">
    <div class="header_resize">
      <div class="logo">
        <h2><a href="index.html">SECURE ACCESS WITH HIDDEN PASSWORD
ENCRYPTION</a></h2>
      </div>
      <div class="clr"></div>
      <div class="menu_nav">
        <ul>
          <li><a href="index.html">Home</a></li>
          <li><a href="endUserLogin.html">Client</a></li>
          <li class="active"><a href="dataOwnerLogin.html"></a></li>
          <li><a href="webServerLogin.html">WebServer</a></li>
          <li></li>
          <li></li>
        </ul>
      </div>
      <div class="clr"></div>
      <img src="images/main_img.jpg" alt="" width="972" height="313" style="padding:25px 0;"
/>
      <div class="clr"></div>
    </div>
  </div>
  <div class="clr"></div>
  <div class="content">
    <div class="content_resize">
      <div class="mainbar">
        <div class="article">
          <h2><span>WelCome::<span class="style1">
<%=(String)application.getAttribute("owner")%>  </br> Your Decrypted
ENP::<%=(String)application.getAttribute("opass")%></span></span></h2>
          <div class="clr"></div>
```

```
          <img src="images/slide1.jpg" width="613" height="505" alt="" />
          <div class="clr"></div>
          <p>
                    </p>
        </div>
      </div>
      <div class="sidebar">
        <div class="gadget">
          <h2>Sidebar Menu</h2>
          <div class="clr"></div>
          <ul class="sb_menu">
            <li><a href="upLoad.jsp">Upload Data</a></li>
            <li><a href="ownerPermission.jsp">Grant Permission</a></li>
            <li><a href="viewOwnerFiles.jsp">View Files</a></li>
            <li><a href="verify.jsp">Verify Your Data</a></li>
                          <li><a href="verifyp.jsp">Verify Your Password</a></li>
            <li><a href="endUserLogin.html">Log Out</a></li>
          </ul>
        </div></div>
      <div class="clr"></div>
      <div class="fbg"></div>
      <div class="clr"></div>
    </div>  </div>
  <div class="footer">
    <div class="clr"></div>
  </div>
</div>
</body>
</html>
```

CHAPTER – 10

RESULTS / DISCUSSIONS

10.1 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the

Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

TYPES OF TESTING

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional testing

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Testing

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document

Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

TESTING METHODOLOGIES

The following are the Testing Methodologies:

- Unit Testing.
- Integration Testing.
- User Acceptance Testing.
- Output Testing.
- Validation Testing.

Unit Testing

Unit testing focuses verification effort on the smallest unit of Software design that is the module. Unit testing exercises specific paths in a module's control structure to ensure complete coverage and maximum error detection. This test focuses on each module individually, ensuring that it functions properly as a unit. Hence, the naming is Unit Testing. During this testing, each module is tested individually and the module interfaces are verified for the consistency with design specification. All important processing path are tested for the expected results. All error handling paths are also tested.

Integration Testing

Integration testing addresses the issues associated with the dual problems of verification and program construction. After the software has been integrated a set of high order tests are conducted. The main objective in this testing process is to take unit tested modules and builds a program structure that has been dictated by design. The following are the types of Integration Testing:

1) Top Down Integration

This method is an incremental approach to the construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main program module. The module subordinates to the main program module are incorporated into the structure in either a depth first or breadth first manner. In this method, the software is tested from main module and individual stubs are replaced when the test proceeds downwards.

2) Bottom-up Integration

This method begins the construction and testing with the modules at the lowest level in the program structure. Since the modules are integrated from the bottom up, processing required for modules subordinate to a given level is always available and the need for stubs is eliminated. The bottom up integration strategy may be implemented with the following steps:

- The low-level modules are combined into clusters into clusters that perform a specific Software sub-function.
- A driver (i.e.) the control program for testing is written to coordinate test case input and output.

10.2 SCREENSHOTS

Fig- 10.2.1 Home page

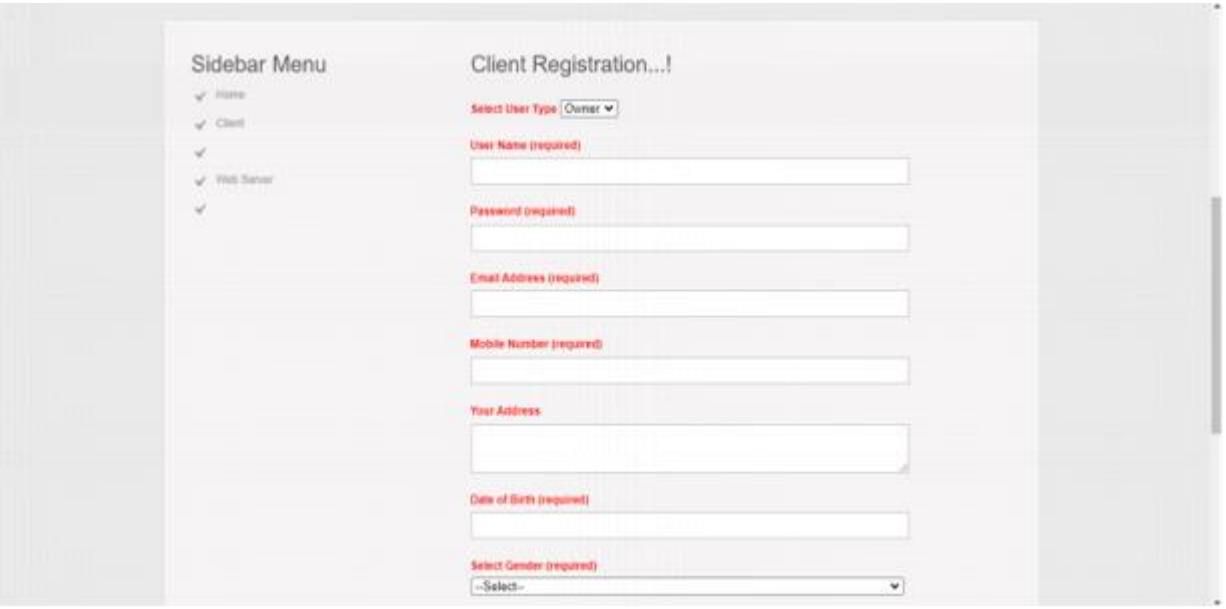


Fig- 10.2.2 Client Registration Page

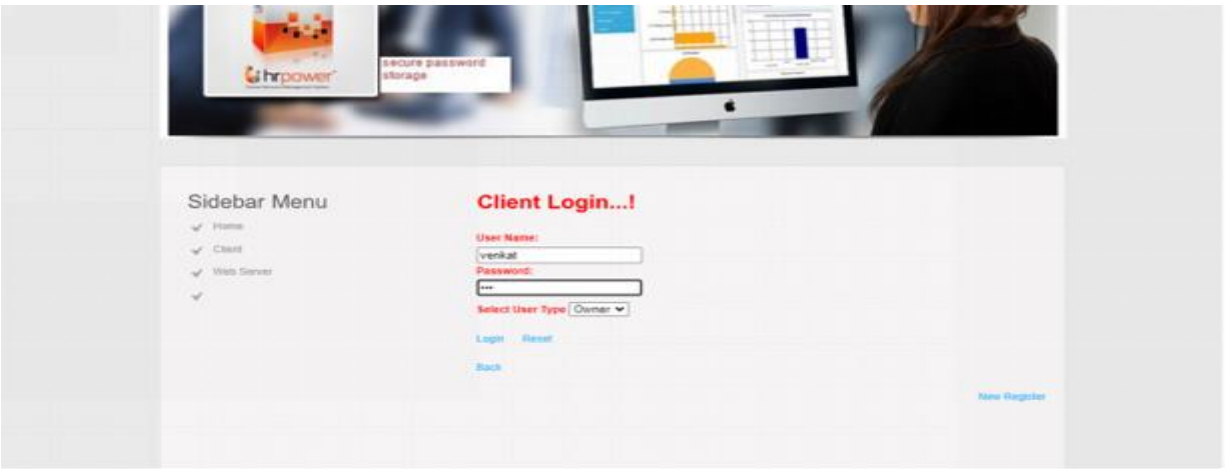


Fig- 10.2.3 Client Login Page

Fig- 10.2.4 Unauthorized Client Home Page

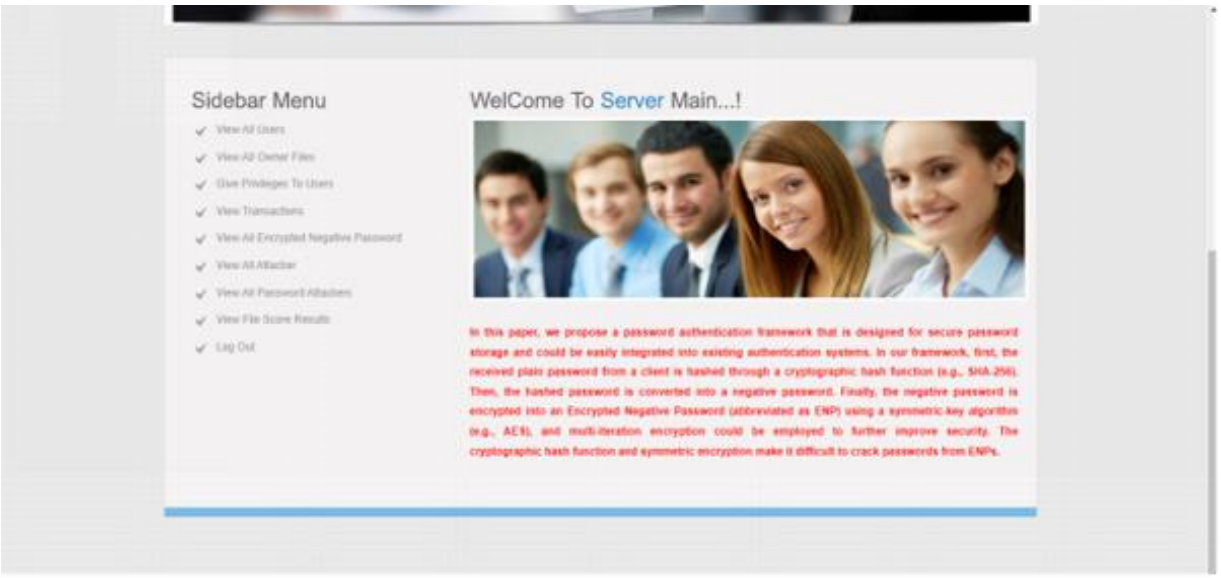


Fig- 10.2.5 Webserver Home Page

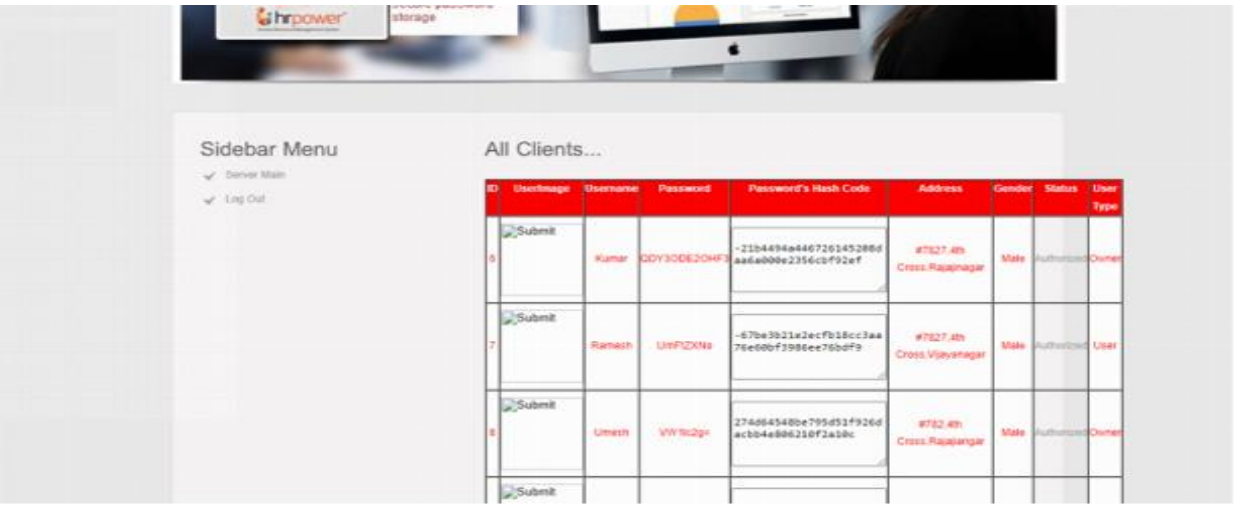


Fig- 10.2.6 All Clients Page

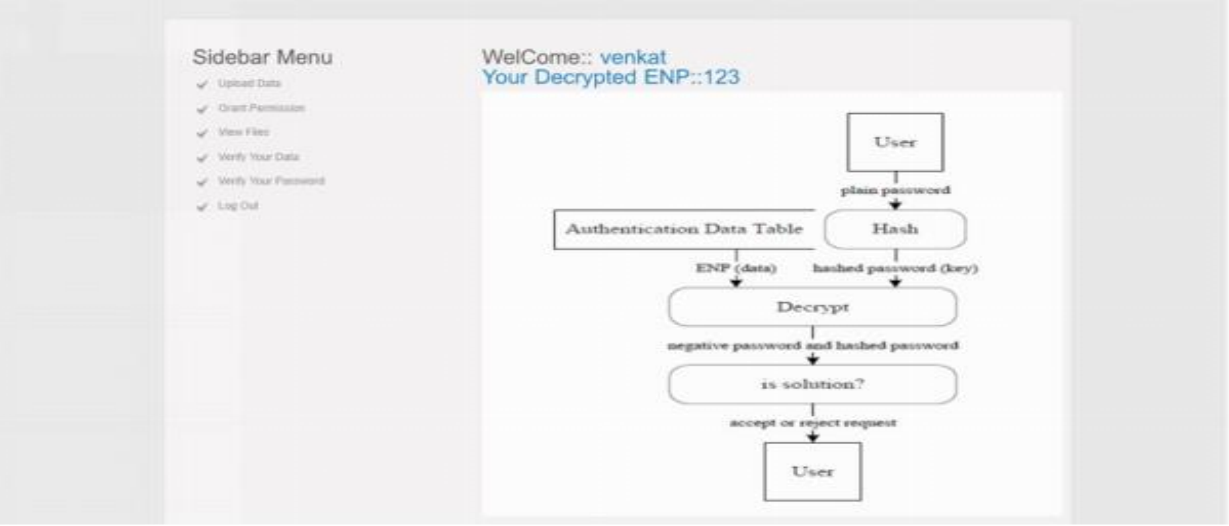


Fig- 10.2.7 Authorized Client Home Page

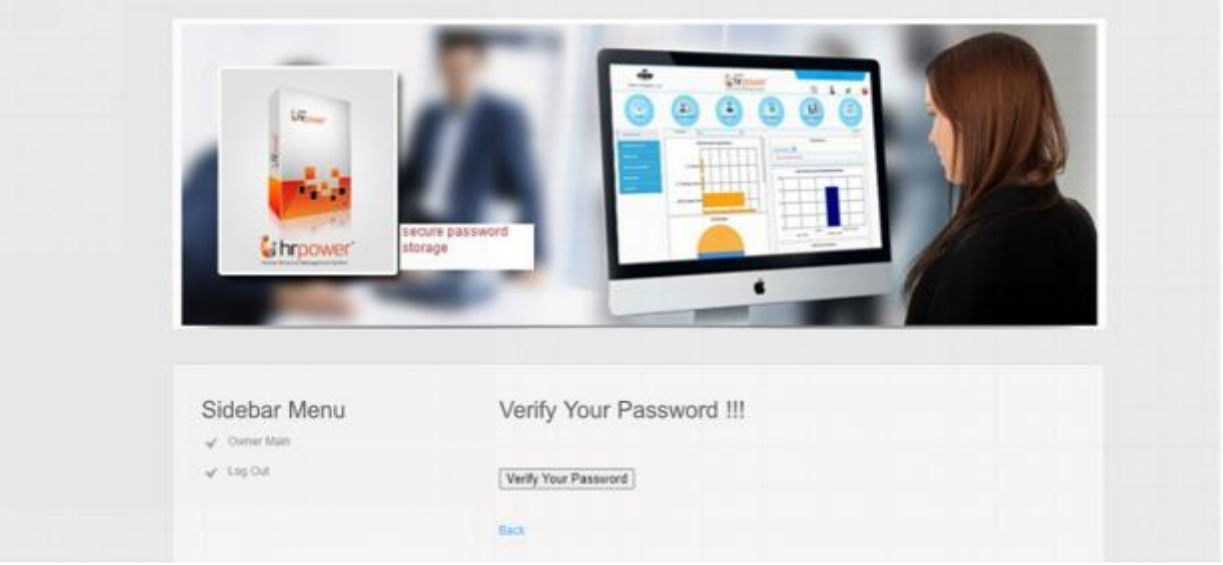


Fig- 10.2.8 Password Verification Page

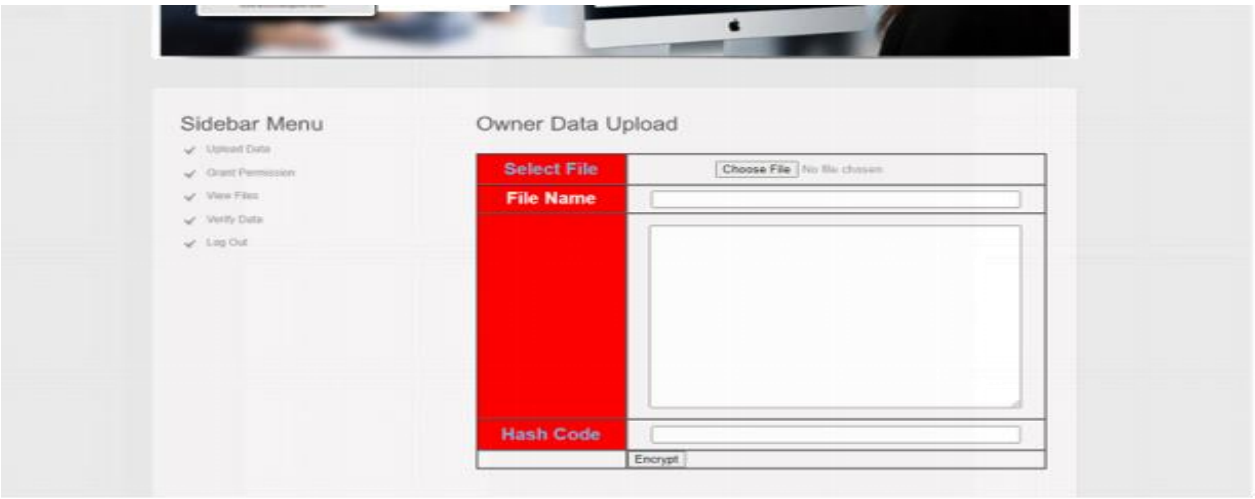


Fig- 10.2.9 Owner Data Upload Page

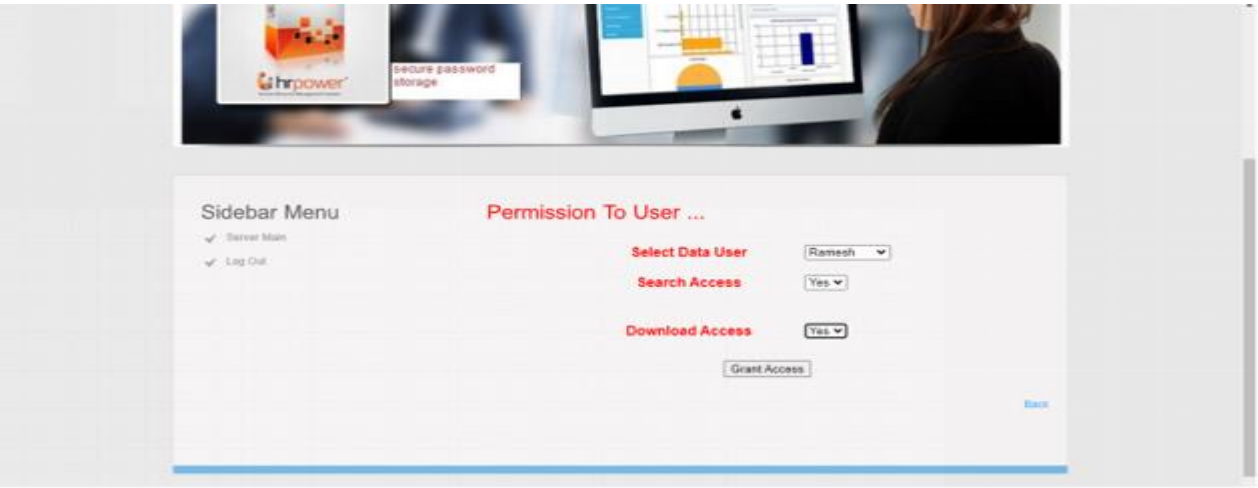


Fig- 10.2.10 Permissions Page

Sl NO	User Name	File Name	Secret Key	Operation	Date & Time
4	Umesh	Connect.jsp	[B@1f4fedf	Upload	05/06/2019 15:21:02
5	Ramesh	Connect.jsp	[B@1f4fedf	Download	05/06/2019 15:24:39
6	Umesh	KeyGen.java	[B@27aaf0	Upload	05/06/2019 15:26:02
7	Umesh	SQL.txt	[B@5d9072	Upload	05/06/2019 15:26:33
8	Manjunath	Android.txt	[B@1d0eefb	Upload	05/06/2019 16:07:27
9	Manjunath	Dotnet.txt	[B@84322	Upload	05/06/2019 16:07:41
10	Manjunath	Java.txt	[B@1b0cc8c	Upload	05/06/2019 16:07:52
11	Manjunath	Silverlight.txt	[B@9955ab	Upload	05/06/2019 16:08:06
12	tmksmanju	Connect.jsp	[B@1f4fedf	Download	05/06/2019 16:10:47
13	tmksmanju	Java.txt	[B@1b0cc8c	Download	05/06/2019 16:12:16
14	tmksmanju	Android.txt	[B@1d0eefb	Download	05/06/2019 16:12:32

Fig- 10.2.11 All Clients And Transactions Page

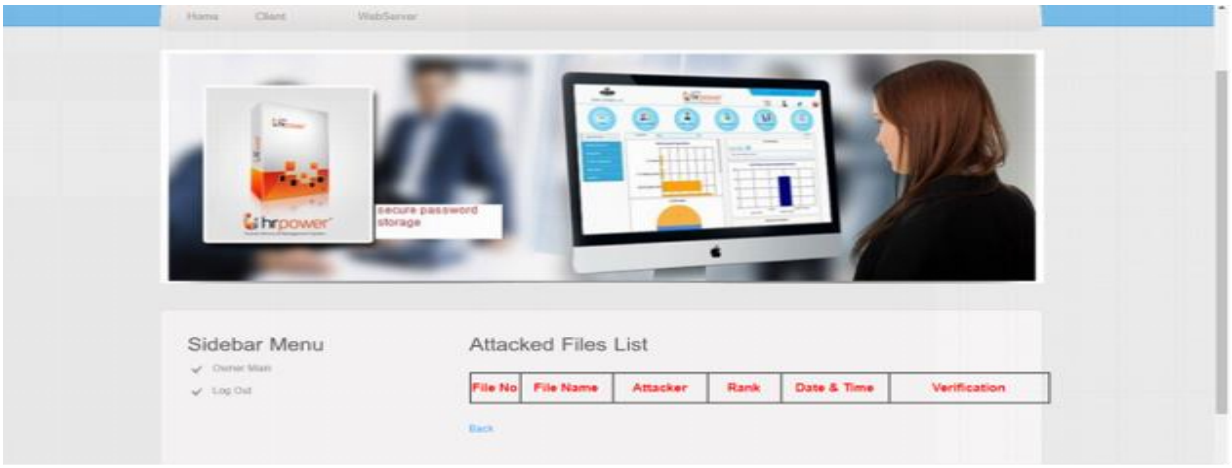


Fig- 10.2.12 Attacked Files Page

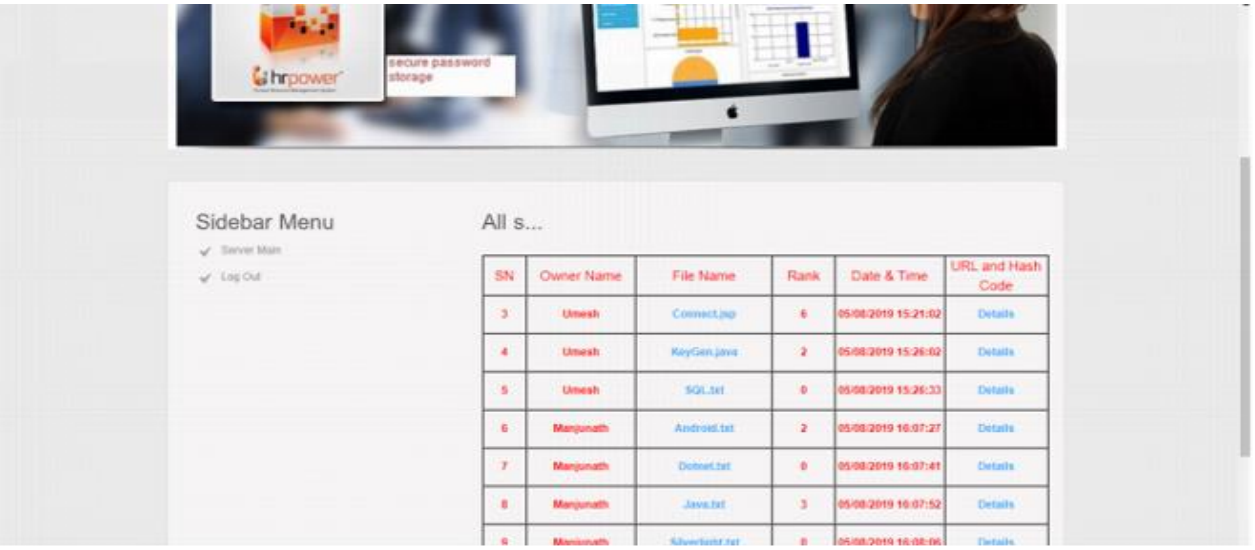


Fig- 10.2.13 All Details Page

Fig- 10.2.14 Attacker Details

Fig- 10.2.15 File Safety Score

CHAPTER – 11

CONCLUSION

We proposed a novel password protection scheme called **Encrypted Negative Password (ENP)** and introduced a comprehensive password authentication framework based on this scheme. Unlike traditional password storage techniques, our framework uses ENP entries within the authentication data table, eliminating the need for additional elements like salts. The design of ENP is both lightweight and secure, offering a practical solution for modern authentication systems.

We conducted a detailed analysis and comparison of the attack complexity between ENP and other common techniques, including hashed passwords, salted passwords, and key stretching methods. The results clearly demonstrate that ENP can effectively resist **lookup table attacks** and provides enhanced protection against **dictionary-based attacks**. One of the key advantages of ENP is its ability to ensure security without requiring external mechanisms or random data.

Looking ahead, we plan to explore the integration of more advanced **Negative Database (NDB)** generation algorithms to further enhance ENP's resistance to sophisticated attacks. Additionally, we aim to incorporate **multi-factor authentication (MFA)** and **challenge–response mechanisms** into our framework to strengthen user verification and provide layered security. These future enhancements are expected to make our ENP-based authentication system more robust and suitable for deployment in high-security applications.

CHAPTER – 12

REFERENCES

[1] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, “Passwords and the evolution of imperfect authentication,” Communications of the ACM, vol. 58, no. 7, pp. 78–87, June. 2015.

[2] M. A. S. Gokhale and V. S. Waghmare, “The shoulder surfing resistant graphical password authentication technique,” Procedia ComputerScience, vol. 79, pp. 490–498, 2016.

[3] J. Ma, W. Yang, M. Luo, and N. Li, “A study of probabilistic password models,” in Proceedings of 2014 IEEE Symposium on Security and Privacy, May 2014, pp. 689–704.

[4] A. Adams and M. A. Sasse, “Users are not the enemy,” Communications of the ACM, vol. 42, no. 12, pp. 40–46, Dec. 1999.

[5] E. H. Spafford, “Opus: Preventing weak password choices,” Computers & Security, vol. 11, no. 3, pp. 273–278, 1992.

[6] Y. Li, H. Wang, and K. Sun, “Personal information in passwords and its security implications,” IEEE Transactions on Information Forensics and Security, vol. 12, no. 10, pp. 2320–2333, Oct. 2017.

[7] D. Florencio and C. Herley, “A large-scale study of web password habits,” in Proceedings of the 16th International Conference on World Wide Web. ACM, 2007, pp. 657–666.

[8] R. Shay, S. Komanduri, A. L. Durity, P. S. Huh, M. L. Mazurek, S. M.Segreti, B. Ur, L. Bauer, N. Christin, and L. F. Cranor, “Designing password policies for strength and usability,” ACM Transactions on Information and System Security, vol. 18, no. 4, pp. 13:1–13:34, May 2016.

[9] D. Wang, D. He, H. Cheng, and P. Wang, “fuzzyPSM: A new password strength meter using fuzzy probabilistic context-free grammars,” in Proceedings of 2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, June. 2016, pp. 595–606.

[10] H. M. Sun, Y. H. Chen, and Y. H. Lin, “oPass: A user authentication protocol resistant to password stealing and password reuse attacks,” IEEE Transactions on Information Forensics and Security, vol. 7, no. 2, pp. 651–663, Apr. 2012.